

# 上下文工程：预算、裁剪、缓存与组装

语言策略：code (Java / Python / TypeScript)

## TL;DR

1. Agent 效果瓶颈常在 Context Engineering，不在模型。
2. 每一轮喂给模型的上下文应当是**预算内、按优先级排序、边界清晰**的组装结果。
3. 上下文工程 = 预算 + 选择 + 压缩 + 组装 + 缓存 + 版本化。

## 1. 上下文预算公式

可用上下文 = 模型窗口 - 保留输出 Token - System Prompt - 工具 Schema  
每轮动态预算 = 可用上下文 - 历史对话预算 - 检索预算 - 记忆预算 - 安全余量

规则：保留输出 Token 按任务最大回复长度计算；工具 Schema 只注入本轮白名单工具；安全余量默认留 10%。

## 2. 内容优先级

| 优先级 | 内容             | 处理策略                |
|-----|----------------|---------------------|
| P0  | 任务目标、验收标准、安全规则 | 永不被裁剪               |
| P1  | 最近 N 轮观察       | 保留最近，更早做摘要          |
| P2  | 工具返回           | 截断长文本，保留结构化字段       |
| P3  | 检索片段           | 只保留 rerank 后的 top-k |
| P4  | 历史记录           | 按相关度注入，低相关不注入       |

## 3. 压缩与裁剪

- 工具返回先提取结构化摘要，再决定是否保留原文；
- 历史对话超过预算时，把早期轮次压缩成状态摘要；
- 代码和日志用行号范围截断，不从中部硬切；
- 外部内容统一包裹为数据区，防止指令注入；
- 达到 40% 上下文利用率后，优先裁剪而不是继续塞入。

## 4. 缓存与版本化

- System Prompt 固定部分放前缀，命中 Prompt Cache；
- 工具 Schema 按 toolset\_version 缓存；
- 检索缓存键 = query 归一化 + 知识库版本 + 租户；
- 每次上下文模板变更要 bump prompt\_version，并跑评测回归。

## 5. 组装顺序

System Prompt (版本化)  
→ 任务卡与验收标准  
→ 本轮白名单工具 Schema  
→ 相关记忆 (低相关不注入)  
→ 检索片段 (标注来源)  
→ 最近观察 (按预算截断)  
→ 用户当前输入 (隔离包裹)

## 6. Python 版上下文组装器

```
1 from dataclasses import dataclass
2
3 @dataclass(frozen=True)
4 class ContextBudget:
5     window_tokens: int
6     reserved_output: int
7     system_prompt: int
```

```
8     tool_schemas: int
9     safety_reserve: int
10
11     def available(self) -> int:
12         return (self.window_tokens - self.reserved_output
13               - self.system_prompt - self.tool_schemas - self.
14               ↪ safety_reserve)
15
16 @dataclass(frozen=True)
17 class ContextSection:
18     name: str
19     priority: int
20     tokens: int
21     content: str
22
23 class ContextAssembler:
24     @staticmethod
25     def assemble(budget: ContextBudget, sections: list[
26     ↪ ContextSection], max_tokens: int) -> str:
27         cap = min(max_tokens, budget.available())
28         parts: list[str] = []
29         used = 0
30         for section in sorted(sections, key=lambda s: s.priority):
31             if used + section.tokens > cap and section.priority > 1:
32                 continue
33             parts.append(f"<!-- section:{section.name} -->\n{section
34             ↪ .content}")
35             used += section.tokens
36         return "\n".join(parts)
```

## 7. 上线检查单

- 每次调用都计算预算，不超模型窗口；
- 工具 Schema 只注入白名单工具；
- 长文本按优先级裁剪，不丢失任务目标；
- 外部数据有数据边界标记；
- Prompt 版本与评测结果绑定；
- 缓存命中率、单任务 Token、上下文利用率有监控。

## 8. 延伸阅读

- [Agent 记忆系统](#)
- [大模型网关](#)
- [Agent 评测体系](#)